INSTITUTO SUPERIOR TÉCNICO
Departamento de Engenharia Informática – Platforms for AI Systems (PSIA) – MEIC, METI – 2026/2027

AI-powered Movie Recommendation Hub

PSIA Project Report - Group 37

João Sérgio Abreu Viegas 109685
Diogo Manuel Candeias da Costa Lobo 109293
Miguel Ângelo Gonçalves da Silva 120490

Demo video: https://youtu.be/xeaE3KXLXQw **Repository:** https://gitlab.rnl.tecnico.ulisboa.pt/psia/groups26/g37

Abstract. We present an AI-powered movie recommendation hub developed for the *Platforms for AI Systems* course. The application combines Spring Boot microservices (authentication, profiles, recommendations and a background agent), a Vue frontend and an Ollama-served LLM that ranks movies from an IMDb-derived catalogue. It is deployed on Google Cloud Platform (GCP) using Infrastructure as Code and monitored with Prometheus and Grafana. As advanced component, a complete CI/CD pipeline tests, builds and deploys every commit. Load tests with up to 100 concurrent clients show throughput levelling off at about 47 req/s while latency keeps growing with the number of clients.

1 Introduction

This project was developed for the Platforms for AI Systems (PSIA) course, which teaches the design, provisioning, deployment and management of a microservices-based application with AI components on a public cloud. Our application is Letterslopd, an AI-powered movie recommendation hub: users register, create a profile with their tastes and receive personalised recommendations ranked by a large language model (LLM) over a catalogue of 7,288 IMDb movies. The main challenge we faced was that the LLM inference is slow, so feeds are pre-computed in the background and cached.

For the Base Solution we built a Spring Boot + Vue application split into four microservices (auth, profile, recommendation and agent), a frontend gateway, an Ollama LLM inference engine and a monitoring stack. It is provisioned with *Terraform*, configured with *Ansible* and deployed on Google Cloud Platform. As Advanced Component we implemented a complete CI/CD pipeline that tests, builds and deploys the app on every commit. A demonstration video is available in the link above as well as the code repository.

The remainder of this report presents the System Design (Section 2), the Advanced Component (Section 3), an Evaluation of throughput and latency as the number of concurrent clients grows (Section 4), and our conclusions (Section 5).

2 System Design and Architecture

2.1 Application Overview

Letterslopd lets a user register and log in, build a profile with favourite genres and liked/disliked movies, search the catalogue (by title, director or cast, filtering by genre, year and rating) and browse other users' profiles. Two AI features sit on top of this: Ask AI, where the user picks a few genres, a mood and optionally describes what they want, and the LLM returns five picks with a short summary on demand, and a personal Feed, pre-computed by a background agent so that fresh recommendations are already waiting when the user logs in.

2.2 Architecture

The user only ever talks to the gateway, an nginx container that serves the Vue build and reverse-proxies `/api/*` to the four Spring Boot microservices (Figure 1). Authentication is stateless (for simplicity): `auth-service` issues a JWT that every other service validates with a shared secret, so no service calls back to it on each request. Services talk to each other over internal REST endpoints (`/internal/*`) that the gateway never routes, and only `recommendation-service` and `agent-service` call the LLM. The main flows are:

- **Ask AI:** `gateway` → `recommendation-service`, which fetches the caller's profile from `profile-service`, builds a shortlist from PostgreSQL and asks Ollama to rank it.
- **Feed:** `gateway` → `agent-service`, which returns the feed cached in Redis if it was built from the current profile version otherwise it answers instantly with a heuristic ranking and starts an AI build in the background.
- **Profile update:** `gateway` → `profile-service` the agent notices the change on its next tick and rebuilds that user's feed.

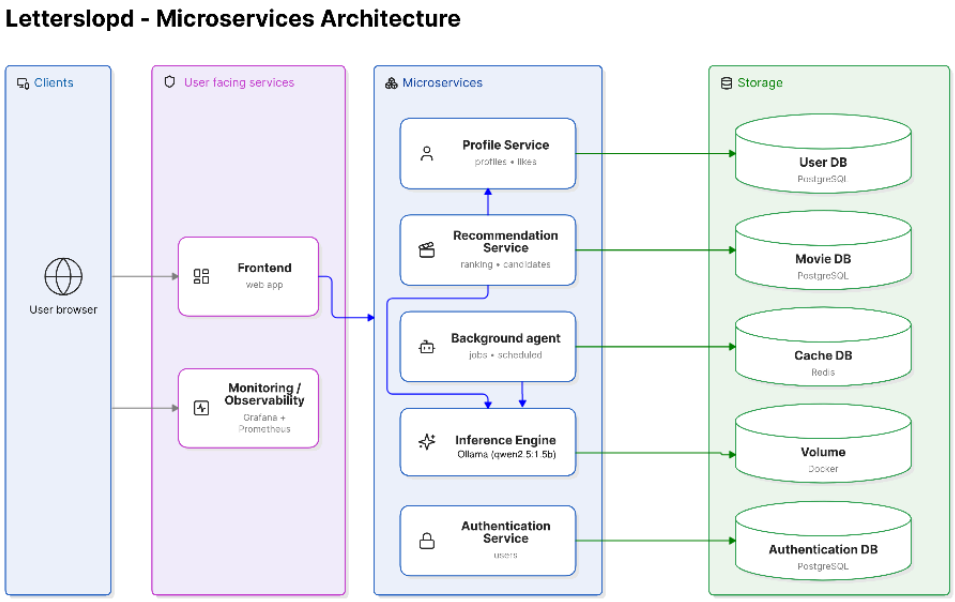


Figure 1: High-level Architecture of the solution.

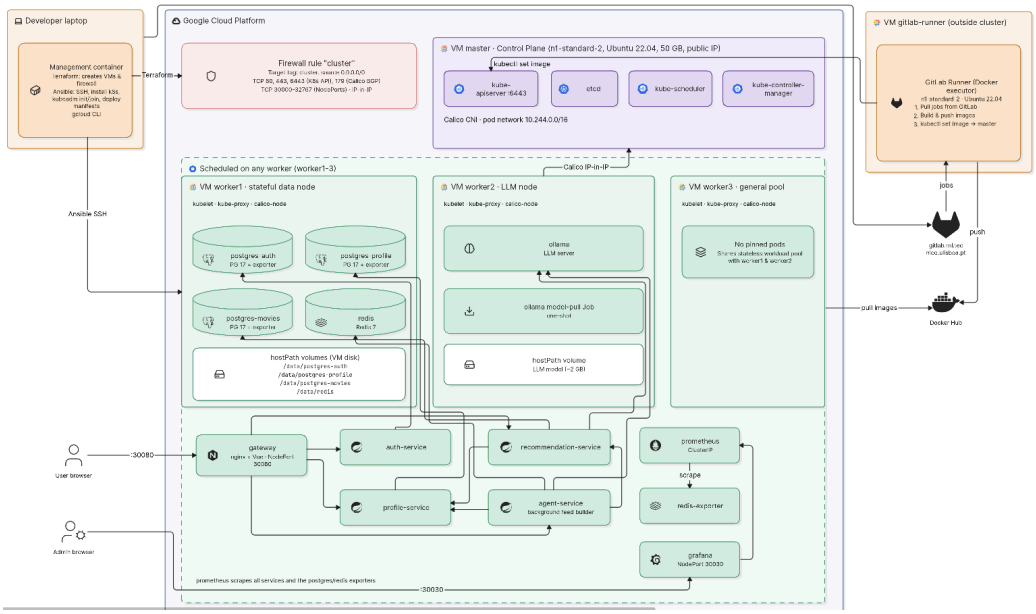


Figure 2: Detailed architecture of the solution.

2.3 Microservices and Data Stores

Each service that needs relational storage gets its own PostgreSQL server (`postgres-auth`, `postgres-profile`, `postgres-movies`), with its own volume and credentials, so they are physically separated. The agent keeps its state (feeds and the last run timestamp) in Redis with append-only persistence.

Table 1: Microservices of the base solution.

Service	Responsibility	Stack	Data store
`gateway`	Serves the Vue build and proxies `/api/*`	nginx, Vue 3 + Vite	–
`auth-service`	Registration, login, JWT issuing	Spring Boot 4, Java 21	PostgreSQL `postgres-auth`
`profile-service`	User profiles and browsing other users	Spring Boot 4, Java 21	PostgreSQL `postgres-profile`
`recommendation-service`	Movie catalogue, LLM-ranked recommendations	Spring Boot 4, Spring AI	PostgreSQL `postgres-movies`
`agent-service`	Background agent that pre-computes feeds	Spring Boot 4, Spring AI	Redis
`llm`	Model inference engine	Ollama	model volume

Service	Responsibility	Stack	Data store
`postgres-*`	One server per service, owned by that service's role	PostgreSQL 17	volume
`redis`	Feed cache and last run timestamp for the agent	Redis	volume

2.4 Dataset

The catalogue comes from the IMDb non-commercial datasets, filtered to the 7,288 movies with at least 25k votes and enriched with TMDB posters and overviews. The CSV is bundled in the `recommendation-service` image and loaded into its database on first startup, so no download is needed at deploy time. New movies can be added and it will work seamlessly.

2.5 AI Components

2.5.1 Inference Engine

We serve Qwen2.5 1.5B with Ollama. Our VMs have no GPU, so a small model is the only one that answers in seconds on CPU. Ollama keeps the model in memory and serves two requests in parallel, so a single background feed build does not block an interactive call. Both AI services call it through Spring AI with structured JSON output and a timeout (25 s for Ask AI, 90 s for background builds).

2.5.2 Recommendation Engine

Recommendations are made in two steps so the UI never depends on the LLM. A content-based scorer first ranks a genre-filtered pool of the catalogue in milliseconds (genre affinity, weighted rating, popularity, shared directors or cast, release era). The LLM then picks 5 of the top 12 candidates and writes a short summary. Picks that are not in the shortlist are discarded, and if the call fails or times out the user gets the heuristic top 5.

2.5.3 Background Agent

Every 120 s the agent fetches the profiles updated since its last run and rebuilds their feeds: LLM picks from the user's favourite genres, plus a section with movies liked by similar users. Feeds are cached in Redis for 24 h and tagged with the profile version, so a stale feed is rebuilt on demand, and at most one build per user runs at a time.

2.6 Infrastructure and Deployment

We deploy on Google Cloud Platform. *Terraform* provisions the firewall rules and one master and three workers (`n1-standard-2`: 2 vCPU, 7.5 GB RAM, Ubuntu 22.04). Five *Ansible* playbooks, run from a management container that bundles all the tooling, then configure the nodes, install containerd and Kubernetes v1.28, create the cluster with `kubeadm` and Calico, join the workers and apply our manifests.

The application is described by plain Kubernetes manifests: a Secret, one Deployment and Service per component, a Job that pulls the model, and the monitoring stack. Bare `kubeadm` has no dynamic storage provisioner, so the databases, Redis and Ollama use `hostPath` volumes on pinned nodes, which keeps their data across pod restarts. Users reach the site through the gateway's NodePort (`http://<any-worker-ip>:30080`).

2.7 Monitoring and Observability

Every Spring service exposes HTTP, JVM and connection-pool metrics through Actuator and Micrometer, plus custom ones such as LLM latency by outcome, recommendations by mode (AI, fallback, quick, cached), feed builds and user activity (registrations, logins, profile updates). Prometheus scrapes them every 15 s together with PostgreSQL and Redis exporters. Grafana opens on a provisioned dashboard (Figure 3) that covers service health, request rate and latency, the AI and agent pipelines, the JVMs and the data stores.

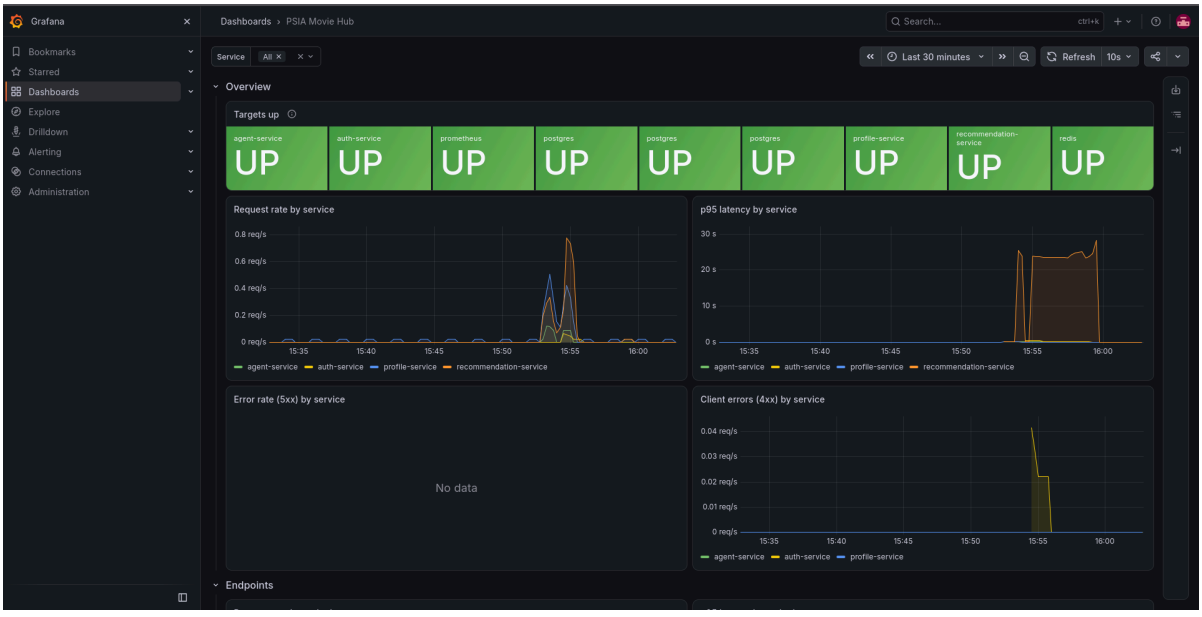


Figure 3: Grafana dashboard showing application monitoring metrics.

2.8 Deployment Instructions

Prerequisites are a GCP project, a service account key, an SSH key pair, a GitLab runner token and a Docker Hub account (the CI/CD pipeline pushes the images). The passwords in the Kubernetes Secret must be set before deploying. Only the Terraform files are GCP-specific, so moving to another cloud means rewriting only those files. The full steps are in the repository's `README.md`.

```
# 1. Provision the infrastructure
terraform init && terraform apply
# 2. Fill in the Ansible inventory (gcphosts) with the IPs from terraform output
# 3. Deploy the application and the monitoring stack
ansible-playbook ansible-gcp-configure-nodes.yml ansible-k8s-install.yml \
  ansible-create-cluster.yml ansible-workers-join.yml ansible-deploy-app.yml -e
docker_hub_user=<DOCKER_HUB_USER>
```

3 Advanced Component: Complete CI/CD Pipeline

3.1 Motivation

Our application is five images that change often, and the cluster is destroyed and recreated between work sessions to save credits. Building, pushing and rolling out each image by hand was slow and easy to get wrong. With the pipeline, every commit to `master` is tested, built, pushed and deployed with no manual steps, and a commit that breaks a test never reaches the cluster.

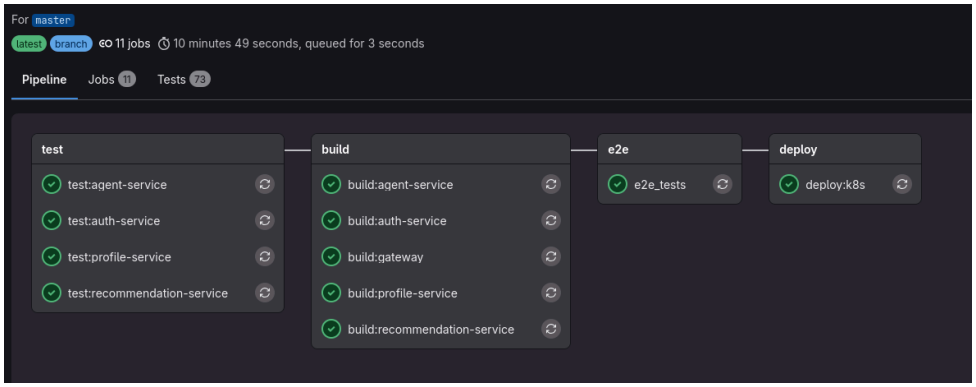


Figure 4: GitLab CI/CD Pipeline overview.

3.2 Design and Implementation

The pipeline runs on GitLab CI in four stages:

1. **test:** Maven unit tests for the four services, up to two at a time.
2. **e2e:** the application is started with Docker Compose on the runner, and Playwright tests drive its main features through a real browser.
3. **build:** each image is built and pushed to Docker Hub tagged with the short commit SHA and `latest`.
4. **deploy:** on `master` only, the job runs `kubectl set image` with the new SHA over SSH and waits for the rolling update. With readiness probes, the new pod starts before the old one stops, and a rollout that is not ready within 300 s fails the pipeline.

Jobs run on a self-hosted GitLab Runner VM, provisioned by the same Terraform code and registered automatically at boot. It uses the Docker executor with the host's Docker socket, which `docker build` and `docker compose` need. Credentials are masked, protected CI variables, and the deploy job only runs while `K8S_MASTER_IP` is set, so the pipeline stays green while the cluster is down.

3.3 Trade-offs and Discussion

A dedicated runner VM was needed because our jobs need a Docker daemon. Mounting the host socket gives jobs root-equivalent access to that VM, which is acceptable only because it runs nothing else. It also costs about as much as one cluster node while it is running. Finally, images are pushed after the e2e stage and that stage rebuilds them from source, so a `latest` image that fails e2e can't reach Docker Hub. The deploy job never rolls it out, since it only uses the SHA-tagged image of a fully green pipeline, but the initial Ansible deploy uses `latest`, so a recreated cluster can start from it.

4 Evaluation

4.1 Experimental Setup

We evaluated the Google Cloud deployment described in Section 2.6, with every component running as a single replica. Load was generated with Apache Bench (`ab`) from a client outside Google Cloud, so every measurement includes the network round trip (about 70 ms for the TCP connection alone).

4.2 Methodology

We benchmarked `POST /api/recommendations/quick`, the most interconnected request path: the gateway forwards it to `recommendation-service`, which validates the JWT, fetches the caller's profile from `profile-`

`service` and its database, and runs the ranking query on the movies database. We left out the LLM-backed endpoint because its responses are cached per user, so repeated requests would mostly measure cache hits, and each uncached call takes seconds on CPU. Each concurrency level ran once with 500 requests, without keep-alive.

4.3 Results: Throughput and Latency

Table 2: Throughput and latency of a request as the number of concurrent clients grows (500 requests per level).

Clients	Throughput (req/s)	p50 (ms)	p95 (ms)	p99 (ms)	Errors (%)
1	5.01	195	246	361	0
2	9.13	209	280	426	0
4	17.63	217	298	384	0
8	29.09	261	373	520	0
16	42.93	328	552	650	0
32	46.89	587	920	1102	0
64	47.70	1185	1958	2548	0
100	45.01	2020	2760	4280	0

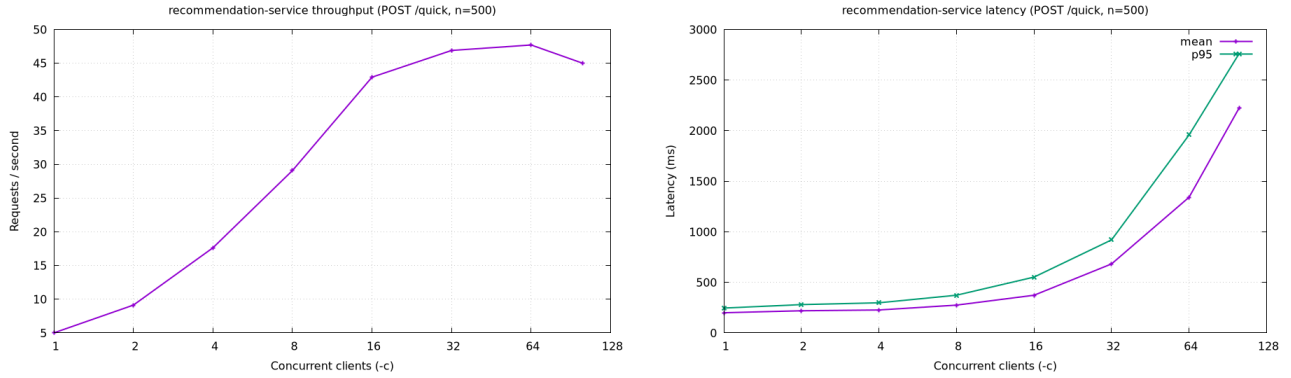


Figure 5: Throughput (left) and mean/p95 latency (right) versus number of concurrent clients (log-scale x axis).

4.4 Discussion

The curves in Figure 5 show the behaviour of a closed system reaching saturation. At low concurrency, throughput grows almost linearly with the number of clients while latency stays close to its baseline of about 200 ms, so the system still has spare capacity and each new client adds useful work. Between 16 and 32 clients throughput bends, and from 32 to 64 clients it stabilises at roughly 47 req/s, which is the maximum the deployment can sustain for this request path.

Past that point, adding clients no longer increases throughput: the extra requests simply wait in a queue, and latency grows roughly in proportion to the number of clients. The tail latency (p95) rises faster than the median, so a minority of users feel the overload much more than the average one. Even so, every request succeeded at all levels, so the system degrades gracefully by slowing down instead of rejecting requests.

Since the connection time stayed constant, the limit is on the server side, in the chain of single-replica services each request crosses. Scaling those services horizontally is the natural way to raise the capacity. The LLM-backed path was not measured. Running on CPU with only two parallel requests, it would saturate far earlier, which is why the design returns an instant ranking first and builds AI feeds in the background.

5 Conclusion

We built and deployed an AI-powered movie recommendation hub that covers every step of the base solution. It is a microservices application with one database per service, packaged with Docker Compose, deployed on a self-hosted Kubernetes cluster on GCP entirely through Terraform and Ansible, and monitored with Prometheus and Grafana. As advanced component, a GitLab CI/CD pipeline tests, builds and deploys every commit to `master`. Under load, the main request path sustains about 47 req/s without errors.

Challenges and lessons learned. The main challenge was running an LLM on CPU-only VMs. A small model, a two-step recommendation with a heuristic fallback and background feed building keep the UI responsive even when the model is slow. Bare `kubeadm` also lacks what managed services provide, such as dynamic storage, so we used `hostPath` volumes on pinned nodes. Finally, keeping the cluster disposable to save credits pushed us to describe the whole setup as code: Terraform creates the VMs, Ansible builds the cluster and deploys the application, and the pipeline rolls out every new commit. Only a few steps remain manual, such as copying the VM IPs into the Ansible inventory.

Future work. With more time we would add more features to the app itself, more replicas and a Horizontal Pod Autoscaler to raise the throughput ceiling, and ideally a GPU node to serve a larger and better model.